STAT3003: 统计算法基础

优化问题（2）

曾靖

中国科学技术大学管理学院

大纲

大纲

1. 梯度下降法
2. 投影梯度法
3. 近端梯度法
4. 坐标下降法

本节内容来自于《统计计算使用 *R*》第 *11* 章和《统计计算》第六章。

1.1 梯度下降法 (Gradient descent)

- 从当前点 x_t 出发，选取方向 u，再沿该方向移动：

$$x_{t+1} = x_t + \eta u, \qquad \eta > 0.$$

- 整体思想：如何选择单位方向 $\|u\|_2 = 1$，使 f 在相同的微小位移下下降幅度最大？

  当 $\eta > 0$ 充分小时，一阶 Taylor 展开给出

$$f(x_t + \eta u) = f(x_t) + \eta \langle \nabla f(x_t), u \rangle + o(\eta).$$

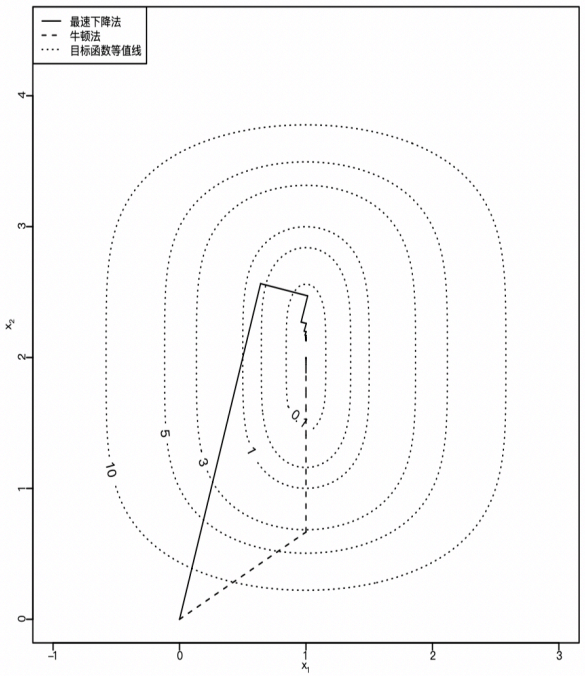
1.1 梯度下降法 (Gradient descent)

- 要使下降幅度最大，就让 $\langle \nabla f(x_t), u \rangle$ 最小。当 $\nabla f(x_t) \neq 0$ 时，取负梯度方向并单位化：

$$u_t = -\frac{\nabla f(x_t)}{\|\nabla f(x_t)\|_2}.$$

- 将归一化因子吸收到步长 s_t 中，得到梯度下降法：

$$\boxed{x_{t+1} = x_t - s_t \nabla f(x_t).}$$



如何选取步长?

- 固定步长: $s_t = s$.
- 最优步长： *limited minimization rule, Armijo or backtracking rule*。

最优步长选择方法

Limited minimization rule: 在给定点 x_t 上，沿着梯度下降方向找到是的函数值最小的步长 $s_t > 0$, 即求解

$$\boxed{s_t = \text{argmin}_{s>0} f(x_t - s\nabla f(x_t)).}$$

最优步长选择方法

Armijo (backtracking) rule: 给定参数 $\alpha \in (0,1)$ 和 $\gamma \in (0,1)$ 以及初始步长 $s = 1$。不断缩短步长 $s \leftarrow \gamma s$ 直至

$$\boxed{f(x_t - s\nabla f(x_t)) \leq f(x_t) - \alpha s \langle \nabla f(x_t), \nabla f(x_t) \rangle,}$$

或等价写为

$$\boxed{f(x_t) - f(x_t - s\nabla f(x_t)) \geq \alpha s \langle \nabla f(x_t), \nabla f(x_t) \rangle.}$$

这里，$s\langle \nabla f(x_t), \nabla f(x_t) \rangle$ 为用一阶近似估计的函数减小量。

- 在一步更新中，函数减小量至少要达到一阶近似减小量的 α 倍。
- 实际应用中，$\alpha = 0.5$ 和 $\gamma = 0.8$ 为合理的取值。

固定步长梯度下降算法

- Input: 函数 f, 初始值 x_0, 步长 s;
- Loop: 对于 $t = 0, 1, ...,$ 重复如下步骤直到 $|f(x_t) - f(x_{t-1})| < \varepsilon$ 或者 $t \geq t_{\max}$:

  Step 1: 更新 $x_{t+1} = x_t - s\nabla f(x_t)$.
- Output: x_t.

最优步长梯度下降算法 *(limited minimization rule)*

- Input: 函数 f, 初始值 x_0
- Loop: 对于 $t = 0, 1, ...,$ 重复如下步骤直到 $|f(x_t) - f(x_{t-1})| < \varepsilon$ 或者 $t \geq t_{\max}$:

  Step 1: 求最佳步长 s_t 使得
  $s_t = \text{argmin}_{s>0} f(x_t - s\nabla f(x_t))$;

  Step 2: 更新 $x_{t+1} = x_t - s_t \nabla f(x_t)$.
- Output: x_t.

例 1.1：一元回归的最小二乘估计

考虑一元回归模型 $y_i = \beta x_i + \varepsilon_i$，$i = 1, \dots, n$。给定样本 (x_i, y_i)，用梯度下降法求最小二乘估计。数值实验中取真实系数 $\beta = 2$。

- 目标函数与最小二乘估计：

$$f(\beta) = \sum_{i=1}^{n} (y_i - \beta x_i)^2, \qquad \hat{\beta} \in \arg\min_{\beta \in \mathbb{R}} f(\beta).$$

- 梯度：

$$f'(\beta) = 2 \sum_{i=1}^{n} x_i (\beta x_i - y_i).$$

- 迭代更新：

$$\beta_{t+1} = \beta_t - s_t f'(\beta_t) = \beta_t - 2 s_t \sum_{i=1}^{n} x_i (\beta_t x_i - y_i).$$

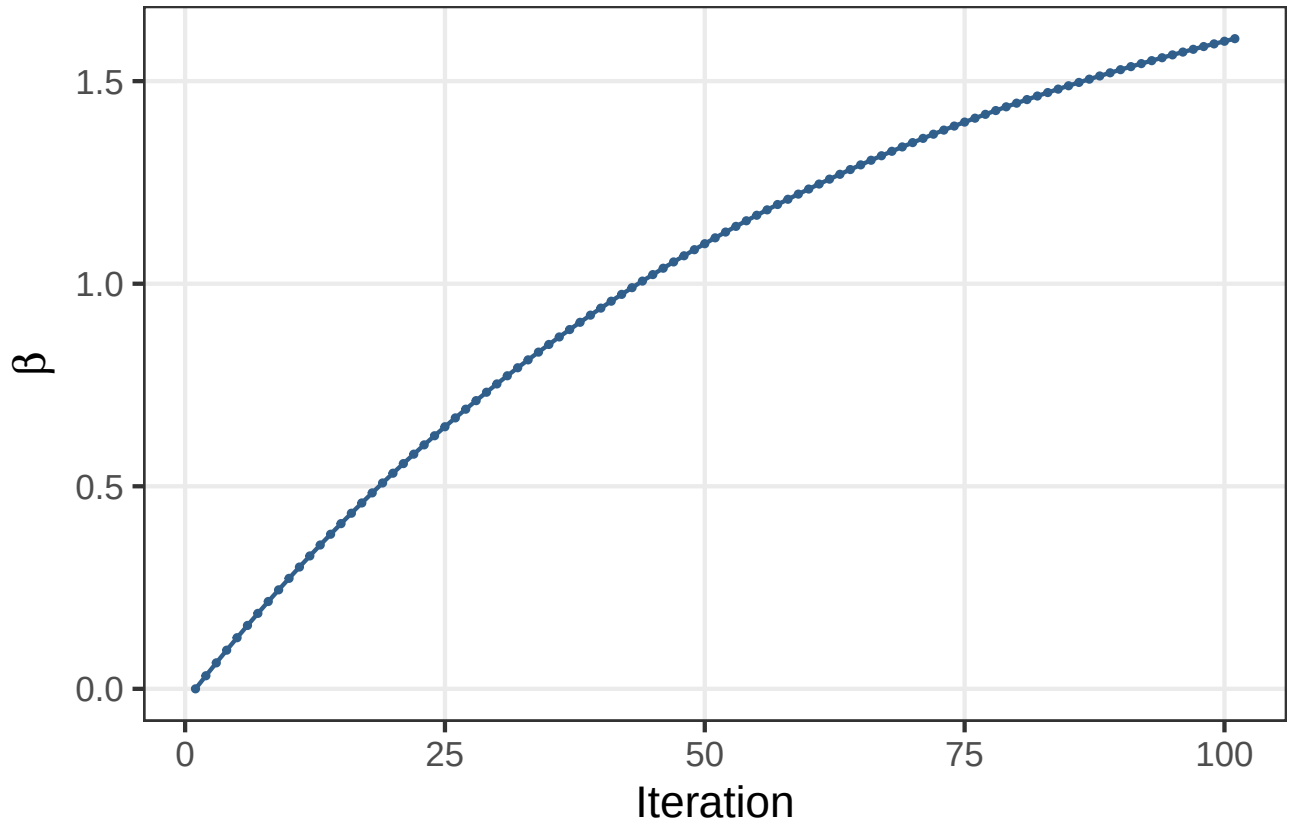
- 固定步长梯度下降法（步长过小）

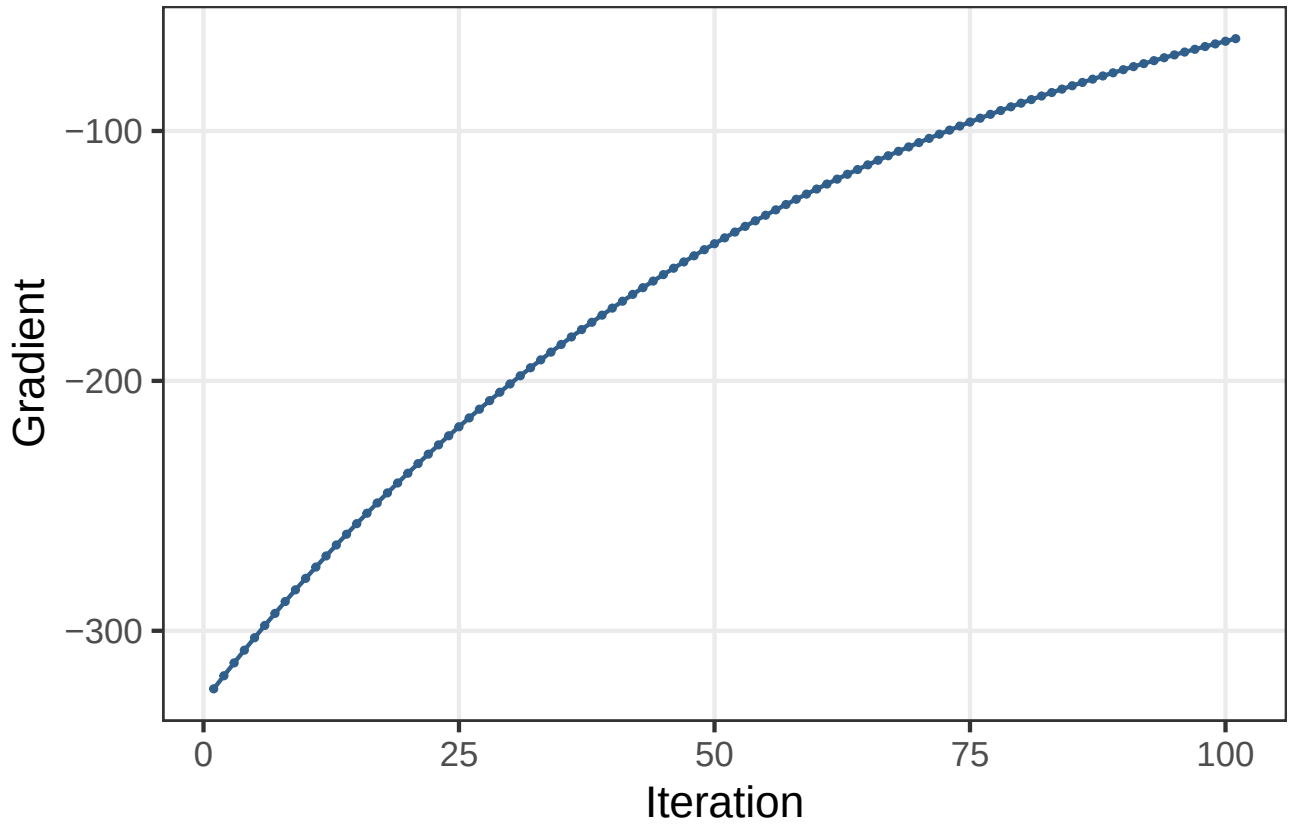
```
n <- 100
x <- rnorm(n)
eps <- rnorm(n)
y <- 2*x + eps
iter <- 0
beta <- c(0)
# Objective function
f <- function(beta){
  sum((y - x * beta)^2)
}
# Gradient function
g <- function(beta){
  2*(sum(x^2)*beta - sum(x*y))
}
```

```r
s <- 1 / 10000 # stepsize
while(abs(g(beta[iter + 1]))>1e-3 & iter < 100){
  iter <- iter + 1
  beta[iter+1] <- beta[iter] - s*g(beta[iter])
}
```

```r
plot_data <- data.frame(
  x = seq_along(beta), value = as.numeric(beta))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "Iteration", y = expression(beta)) +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

```r
plot_data <- data.frame(
  x = seq_along(g(beta)), value =
  as.numeric(g(beta)))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "Iteration", y = "Gradient") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```


β
1.5
1.0
0.5
0.0
0
25
50
75
100
Iteration


Gradient
−100
−200
−300
0
25
50
75
100
Iteration

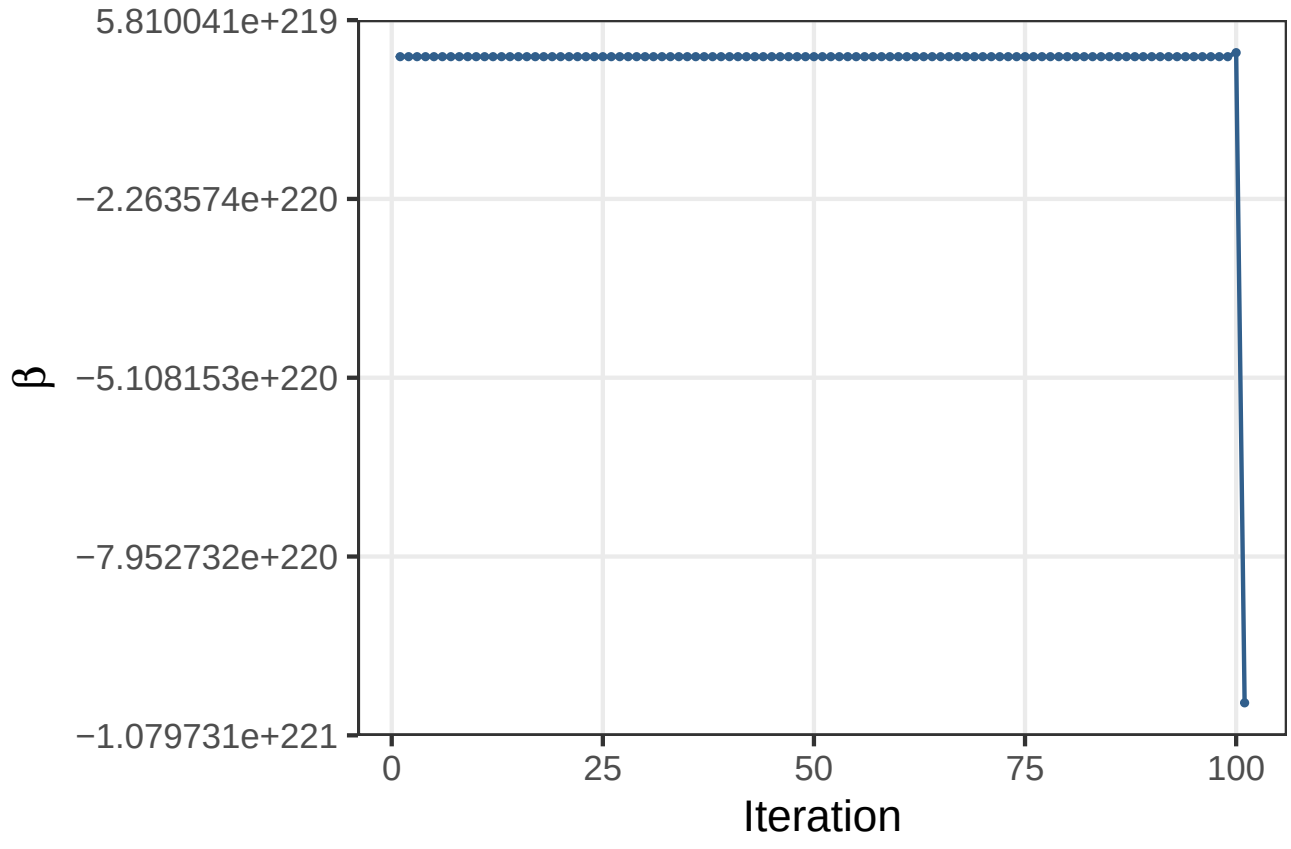
- 固定步长梯度下降法（步长过大）

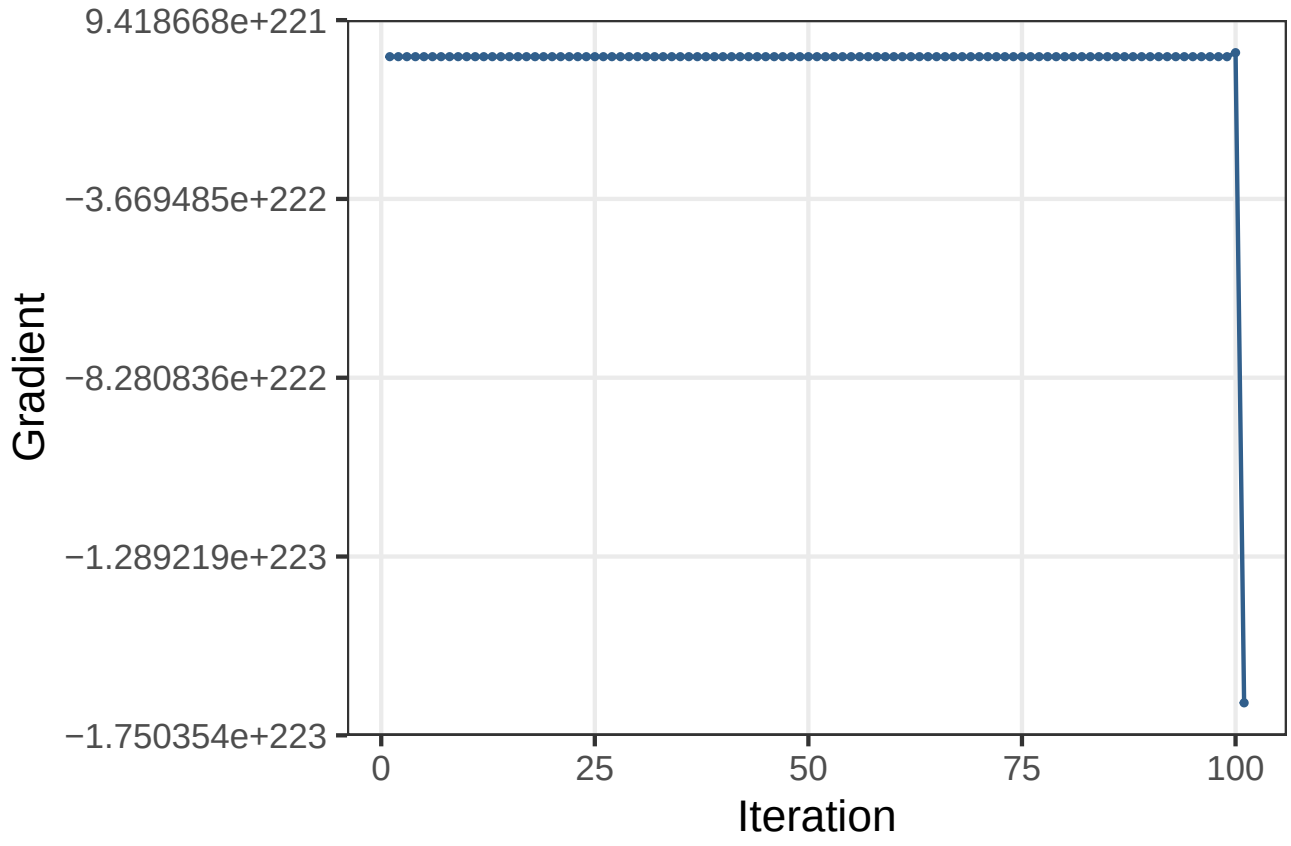
```
iter <- 0
beta <- 0

s <- 1 # stepsize
while(abs(g(beta[iter + 1]))>1e-3 & iter < 100){
  iter <- iter + 1
  beta[iter+1] <- beta[iter] - s*g(beta[iter])
}
```

```r
plot_data <- data.frame(
  x = seq_along(beta), value = as.numeric(beta))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "Iteration", y = expression(beta)) +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

```r
plot_data <- data.frame(
  x = seq_along(g(beta)), value =
  as.numeric(g(beta)))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "Iteration", y = "Gradient") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```


5.810041e+219
−2.263574e+220
−5.108153e+220
−7.952732e+220
−1.079731e+221
β
0
25
50
75
100
Iteration


9.418668e+221
−3.669485e+222
−8.280836e+222
−1.289219e+223
−1.750354e+223
Gradient
0
25
50
75
100
Iteration

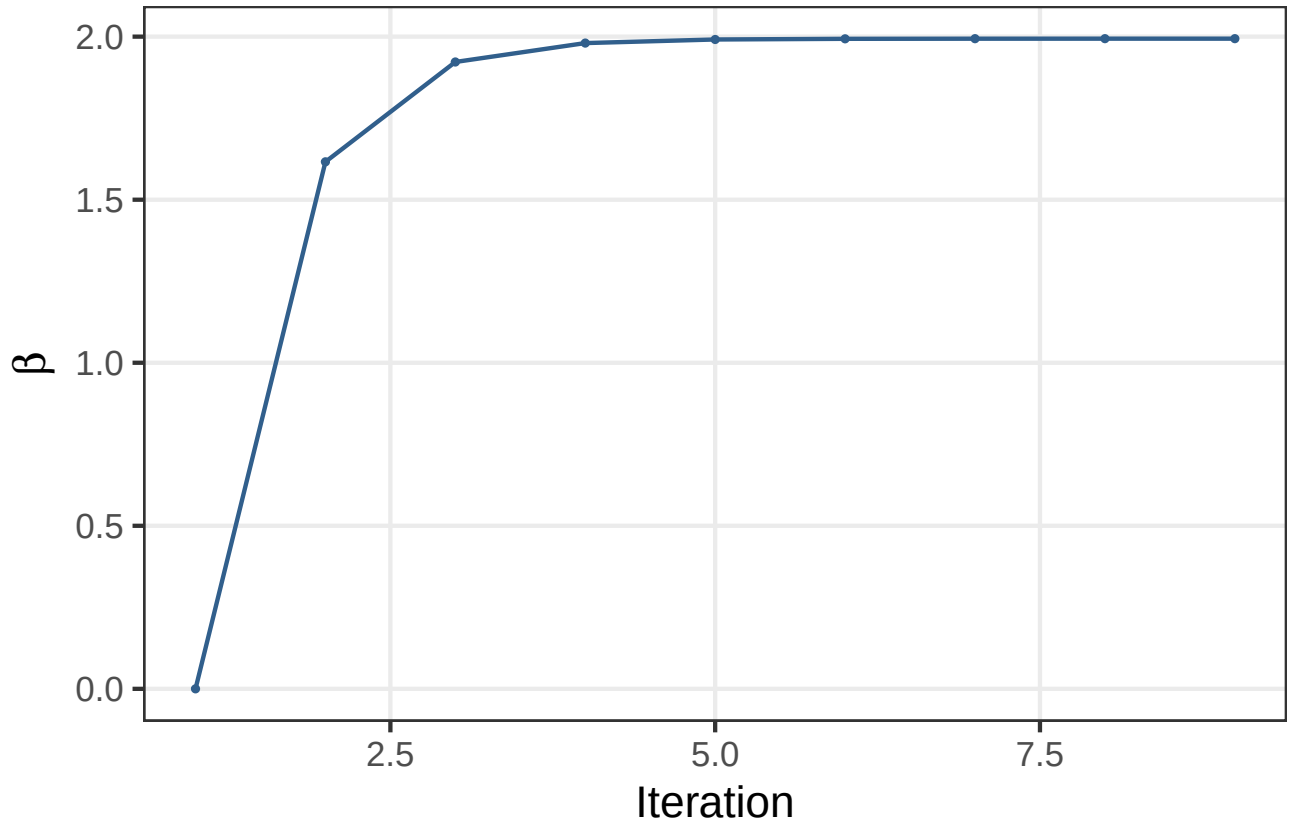
- 固定步长梯度下降法（步长合适）

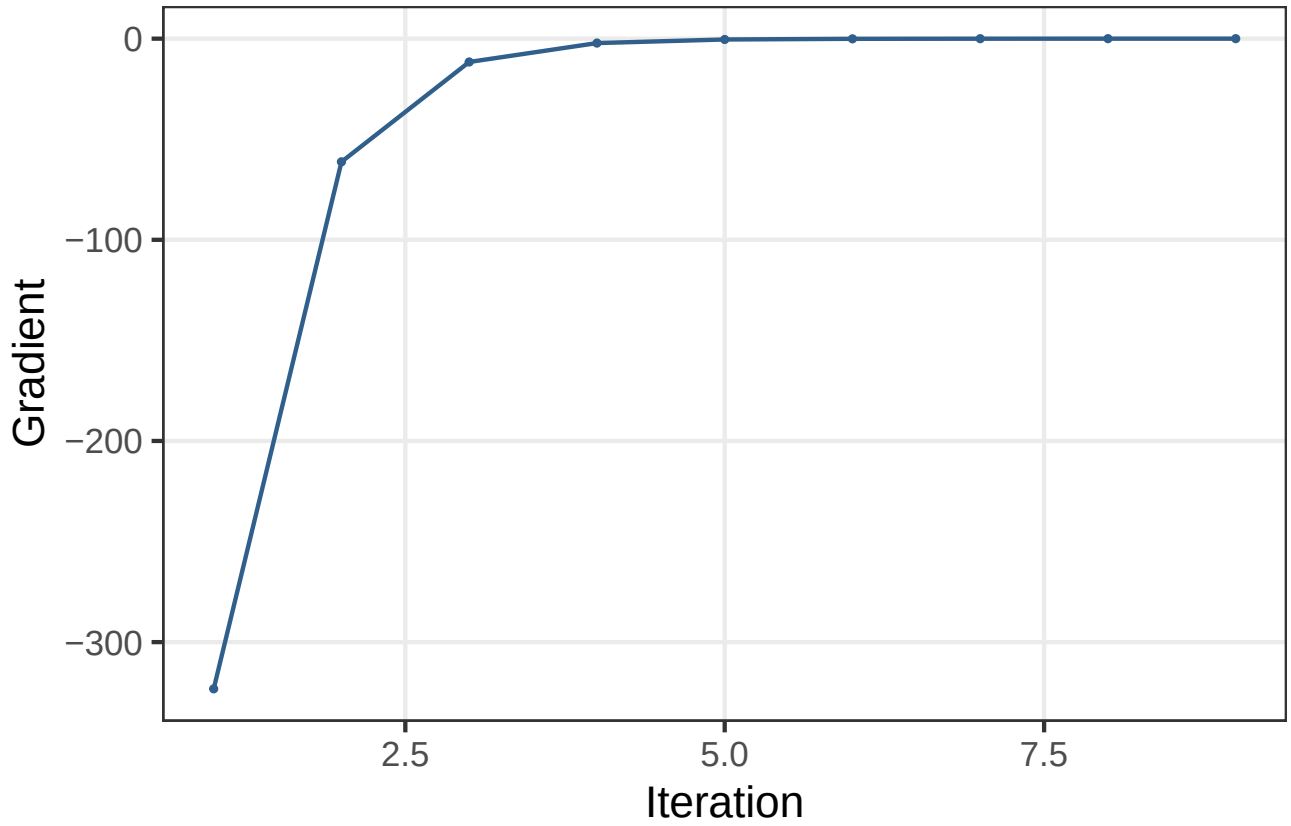
```
iter <- 0
beta <- 0

s <- 1 / 200 # stepsize
while(abs(g(beta[iter + 1]))>1e-3 & iter < 100){
  iter <- iter + 1
  beta[iter+1] <- beta[iter] - s*g(beta[iter])
}
```

```r
plot_data <- data.frame(
  x = seq_along(beta), value = as.numeric(beta))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "Iteration", y = expression(beta)) +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

```r
plot_data <- data.frame(
  x = seq_along(g(beta)), value =
  as.numeric(g(beta)))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "Iteration", y = "Gradient") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```


2.0
1.5
1.0
0.5
0.0
β
2.5
5.0
7.5
Iteration


0
−100
−200
−300
Gradient
2.5
5.0
7.5
Iteration

- 最优步长梯度下降算法 (limited minimization rule)

Question: How to compute the optimal stepsize?

```
iter <- 0
beta <- 0

while(abs(sum(x^2)*beta - sum(x*y))>1e-3 & iter <
100){
  iter <- iter + 1
  s <- 0.5/sum(x^2)
  beta <- beta - s * 2*(sum(x^2)*beta - sum(x*y))
  cat(iter, beta, "\n")
}
```

```
## 1 1.993876
```

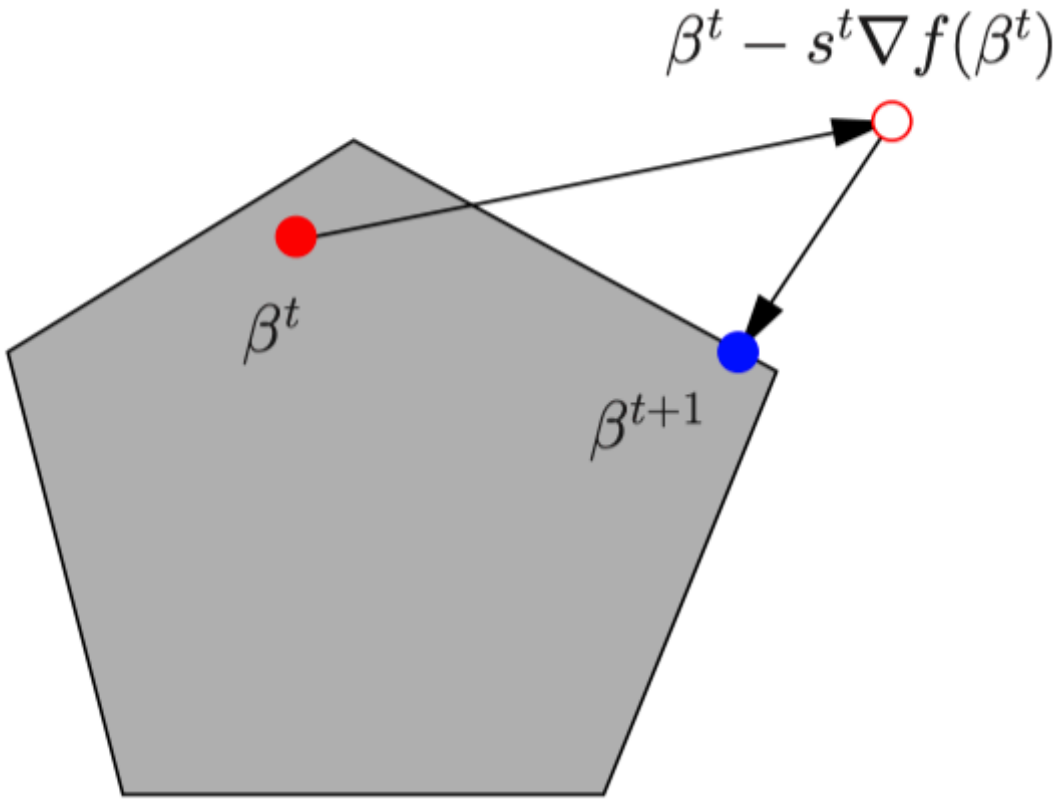
1.2 投影梯度法

- 投影下降法 *(Projected Gradient Method)* 是用来处理具有约束条件的优化算法。
- 考虑带有约束条件的优化问题：

$$x^* = \arg\min_{x \in \mathcal{C}} f(x),$$

这里 $\mathcal{C}$ 为一个约束空间。

投影梯度法的图示



- 回忆梯度下降法的迭代公式为

$$x_{t+1} = x_t - s_t \nabla f(x_t).$$

- 梯度下降法迭代公式的另一种等价写法为

$$\boxed{x_{t+1} = \arg\min_{x \in \mathbb{R}^p} \Big\{ f(x_t) + \langle \nabla f(x_t), x - x_t \rangle + \frac{1}{2s_t} \|x - x_t\|_2^2 \Big\}.}$$

- 投影梯度法的迭代公式为:

$$
\begin{aligned}
x_{t+1} &= \arg\min_{x\in\mathcal{C}} \left\{ f(x_t) + \langle \nabla f(x_t), x - x_t \rangle + \frac{1}{2s_t}\|x - x_t\|_2^2 \right\} \\
&= \arg\min_{x\in\mathcal{C}} \|x - x_t + s_t \nabla f(x_t)\|_2^2 \\
&= \arg\min_{x\in\mathcal{C}} \|x - (x_t - s_t \nabla f(x_t))\|_2^2
\end{aligned}
$$

- 投影梯度法等价于先做一次梯度下降

$$\tilde{x}_{t+1} = x_t - s_t \nabla f(x_t),$$

再将其投影到约束空间上，即

$$x_{t+1} = \arg\min_{x\in\mathcal{C}} \|x - \tilde{x}_{t+1}\|.$$

投影梯度算法

Step 1: 初始化 x_0;

Step 2: 计算 $\tilde{x}_{t+1} = x_t - s_t \nabla f(x_t)$;

Step 3: 更新 $x_{t+1} = \arg\min_{x \in \mathcal{C}} \|x - \tilde{x}_{t+1}\|$.

1.3 近端梯度法

- 近端梯度法 *(Proximal gradient method)* 一般用于凸优化问题中，主要用来处理如下优化问题：

$$x^* = \arg\min_{x \in \mathbb{R}^p} f(x), \quad f(x) = g(x) + h(x),$$

其中 g 函数为凸函数且可微，h 函数为凸函数但不可微。

Lasso problem

将最小二乘损失与 L_1 惩罚相加：

$$\min_{\beta \in \mathbb{R}^p} \underbrace{\frac{1}{2n}\|y - X\beta\|_2^2}_{g(\beta)\text{：凸且可微}} + \underbrace{\lambda\|\beta\|_1}_{h(\beta)\text{：凸但不可微}}, \qquad \lambda > 0.$$

1.3 近端梯度法

- 近端梯度法来源于梯度下降法中迭代公式的等价写法

$$x_{t+1} = \arg\min_{x\in\mathbb{R}^p} \left\{ f(x_t) + \langle \nabla f(x_t), x - x_t \rangle + \frac{1}{2s_t}\|x - x_t\|_2^2 \right\}.$$

- 对于可微部分 g，我们采取同样的近似方法，而对 h 保持不动。迭代公式为

$$x_{t+1} = \arg\min_{x\in\mathbb{R}^p} \Big\{ \underbrace{g(x_t) + \langle \nabla g(x_t), x - x_t \rangle + \frac{1}{2s_t}\|x - x_t\|_2^2}_{g \text{ 的二次近似}} + h(x) \Big\}.$$

- 定义近端映射算子 *(proximal map operator)*:

$$\text{prox}_h(y) = \arg\min_{x \in \mathbb{R}^p} \frac{1}{2}\|y - x\|_2^2 + h(x).$$

- 利用近端映射算子，近端梯度法的迭代公式可以改写为

$$\boxed{x_{t+1} = \text{prox}_{s_t h}(x_t - s_t \nabla g(x_t)).}$$

- 特例：令 $h(x) = I_{\mathcal{C}}(x)$，近端梯度法退化为投影梯度法。

近端梯度算法

Step 1: 初始化 x_0;

Step 2: 计算 $\tilde{x}_{t+1} = x_t - s_t \nabla g(x_t)$;

Step 3: 更新 $x_{t+1} = \text{prox}_{s_t h}(\tilde{x}_{t+1})$.

近端映射算子的一些特殊例子

- *Projection*（投影）：当函数

$$h(x) = I_{\mathcal{C}}(x) = \begin{cases} 0, & \text{if } x \in \mathcal{C}, \\ \infty, & \text{otherwise} \end{cases}$$

则

$$\text{prox}_h(y) = \arg\min_{x \in \mathcal{C}} \|y - x\|_2^2$$

是一个投影算子，将 y 投影到空间 $\mathcal{C}$ 上;

近端映射算子的一些特殊例子

- L_1 *penalty*（L_1 惩罚）：先考虑一元情形

$$h(x) = \lambda|x|, \qquad x \in \mathbb{R}, \quad \lambda > 0.$$

对输入 $y \in \mathbb{R}$，近端映射为

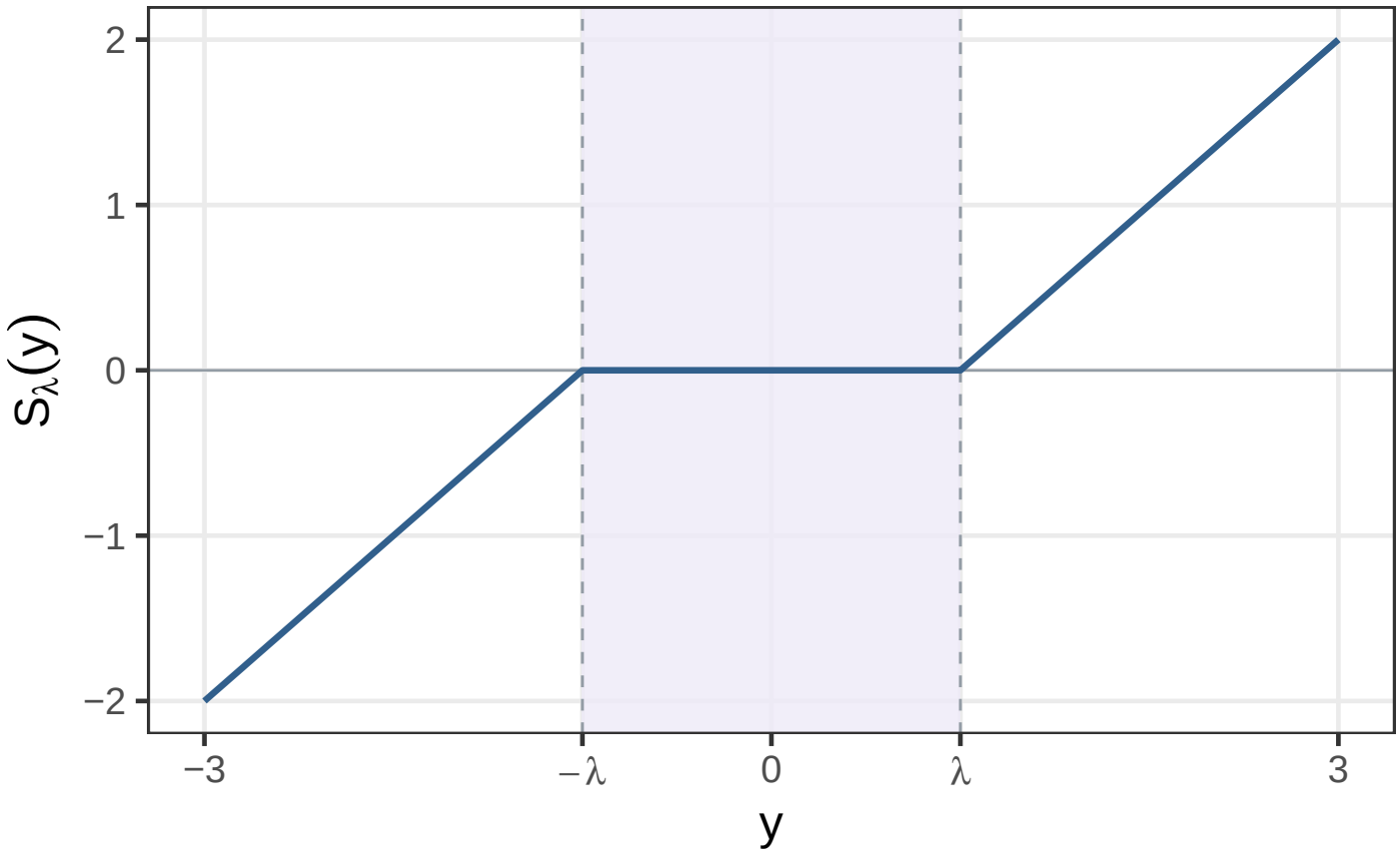
$$\text{prox}_h(y) = \mathcal{S}_\lambda(y).$$

其中，*soft-thresholding operator* 定义为

$$\mathcal{S}_\lambda(y) = \text{sign}(y)(|y| - \lambda)_+,$$

这里 $(a)_+ = \max\{a, 0\}$。

Soft-thresholding operator



- $|y| \leq \lambda$ 时输出为零；其余取值向零收缩 λ。

收敛速率

- 假设目标函数 $f = g + h$, g 函数连续可微并且梯度满足 Lipschitz 连续，即存在常数 $L > 0$,

$$\|\nabla g(x) - \nabla g(x')\|_2 \leq L\|x - x'\|_2, \quad \forall x, x' \in \mathbb{R}^p.$$

  那么当取固定步长 $s_t = s \in (0, 1/L]$，目标函数有如下收敛速率（Nesterov, 2007）:

$$|f(x_t) - f(x^*)| \leq \frac{C}{t+1}\|x_0 - x^*\|_2.$$

- 因此对于性质良好的目标函数，近端梯度法可以保证目标函数具有次线性收敛速率。

文献：Nesterov, Y. (2007). *Gradient methods for minimizing composite objective function*. CORE Discussion Paper 2007/76.

- 如果进一步假设 g 函数具有强凸性（strong convexity），即

$$g(x+\Delta)-g(x)-\langle \nabla g(x),\Delta\rangle \geq \gamma^2\|\Delta\|_2^2, \quad \forall x,\Delta\in\mathbb{R}^p,$$

那么近端梯度法可以保证目标函数达到线性收敛速率，

$$|f(x_t)-f(x^*)|\leq C\kappa^t\|x_0-x^*\|_2,$$

这里 $\kappa\in(0,1)$ 是一个常数。

例 1.2: Lasso problem

对于数据矩阵 $y \in \mathbb{R}^n, X \in \mathbb{R}^{n\times p}$，考虑如下求解问题

$$\min_{\beta\in\mathbb{R}^p} \frac{1}{2n}\|y - X\beta\|_2^2 + \lambda\|\beta\|_1.$$

- $g(\beta) = \|y - X\beta\|_2^2/(2n)$，$h(\beta) = \lambda\|\beta\|_1$。
- 欧氏范数下，最小的 Lipschitz 常数为

$$L = \frac{1}{n}\|X^\top X\|_{\text{op}} = \frac{\lambda_{\max}(X^\top X)}{n}.$$

近端梯度法

- 取固定步长 $s_t = 1/L$, $L = (1/n)\|X^\top X\|_{\text{op}}$.
- 迭代公式为:

$$\beta_{t+1} = \text{prox}_{s_t h}(\beta_t - s_t \nabla g(\beta_t)),$$

Step 1: $\tilde{\beta}_{t+1} = \beta_t - s_t \nabla g(\beta_t) = \beta_t + \frac{1}{n} s_t X^\top (y - X\beta_t)$;

Step 2: $\beta_{t+1,j} = \mathcal{S}_{s_t \lambda}\{\tilde{\beta}_{t+1,j}\}, j = 1, \ldots, p.$

- 低维数据: $p = 8$

```r
# Generate data
set.seed(2)
n <- 500
p <- 8
x <- matrix(rnorm(n*p), n, p)
beta <- c(3, 1.5, 0, 0, 2, 0, 0, 0)
y <- x%*%beta + rnorm(n)

soft <- function(x, lam){
  ifelse(abs(x)>lam, sign(x)*(abs(x)-lam), 0)
}
```

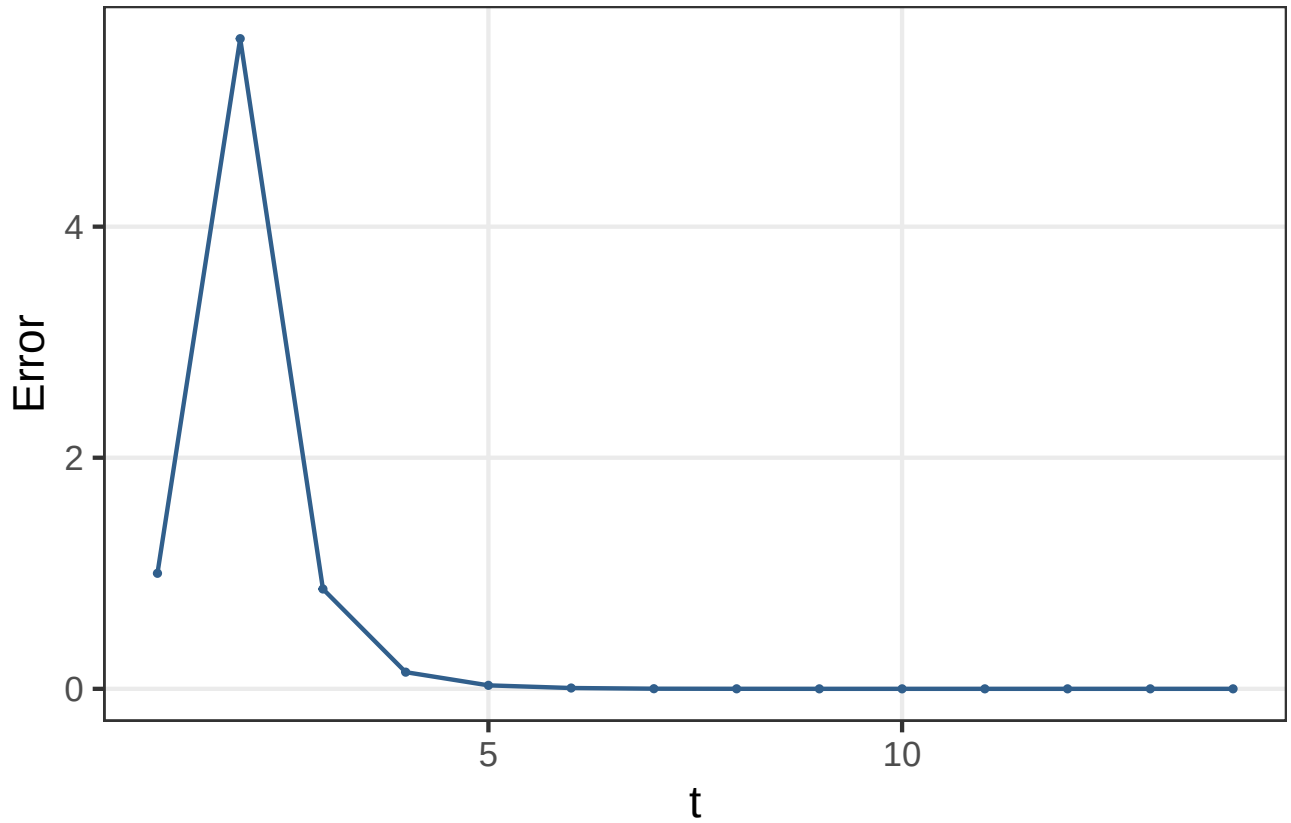
```r
# Initialization
b.old <- b <- rep(0, 8)
cvg <- c(1)
iter <- 0
lambda <- 0.1
tol <- sqrt(.Machine$double.eps)
step <- n/(svd(t(x)%*%x)$d[1])
step
```

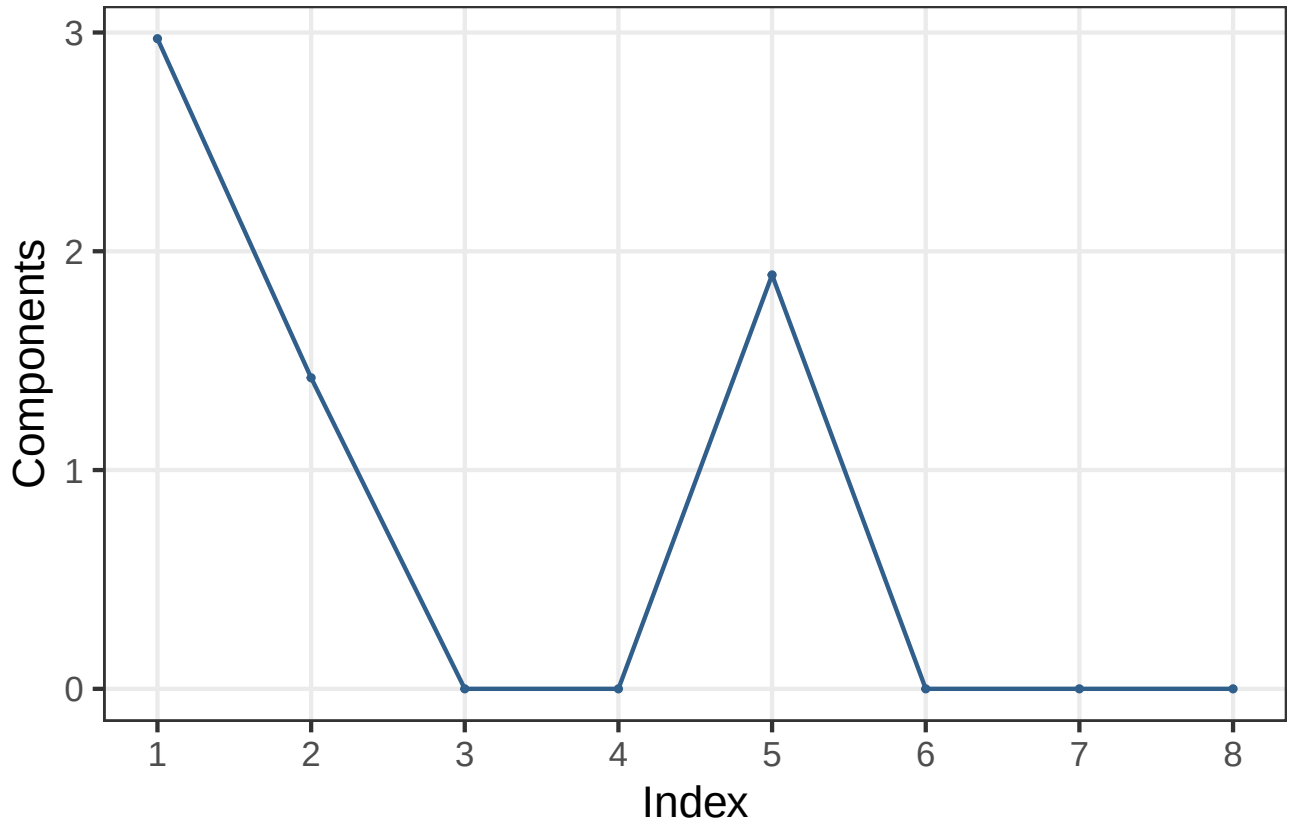
```
## [1] 0.8500063
```

```r
options(width = 50)
while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  b <- b.old + (1/n)*step*t(x)%*%(y - x%*%b.old)
  b <- sapply(b, soft, lambda*step)
  diff <- sum(abs(b-b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```r
plot_data <- data.frame(
  x = seq_along(cvg), value = as.numeric(cvg))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "t", y = "Error") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

```r
plot_data <- data.frame(
  x = seq_along(b.old), value = as.numeric(b.old))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  scale_x_continuous(breaks = 1:8) +
  labs(x = "Index", y = "Components") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```


Error
4
2
0
5
10
t


Components
3
2
1
0
1
2
3
4
5
6
7
8
Index

- 高维数据: $p = 200$

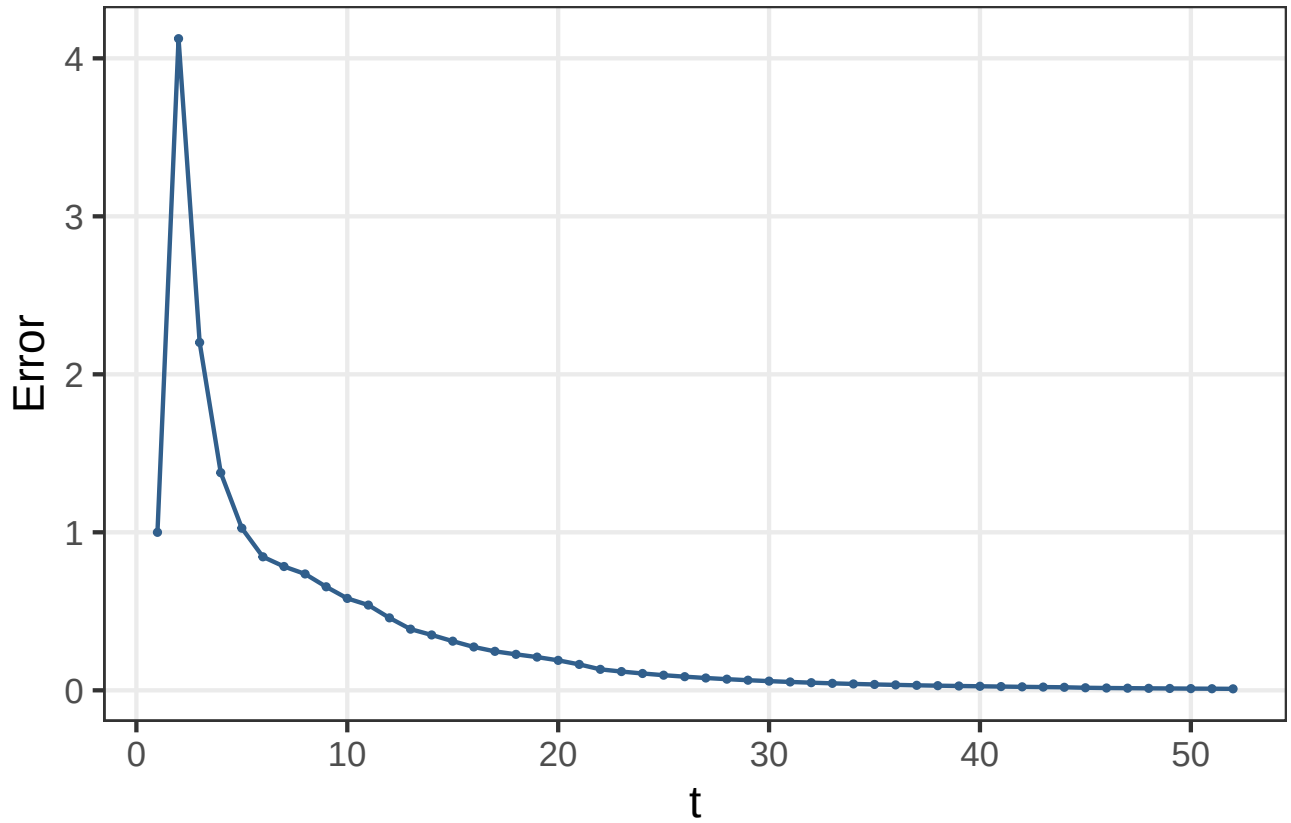
```
set.seed(2)
n <- 100
p <- 200
x <- matrix(rnorm(n*p), n, p)
beta <- rep(0, p)
beta[1:10] <- 2
y <- x%*%beta + rnorm(n)
```

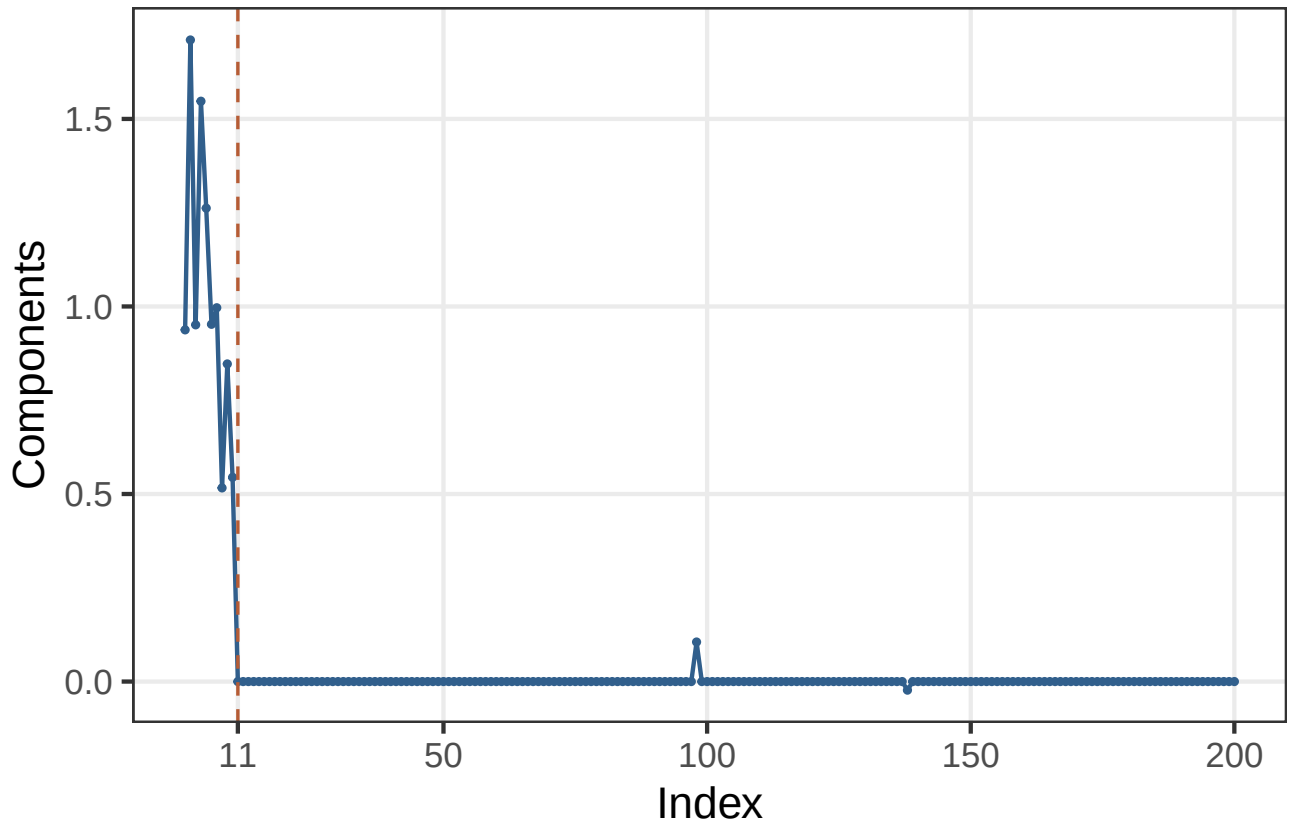
```r
# Initialization
b.old <- b <- rep(0, p)
cvg <- c(1)
iter <- 0
lambda <- 0.8
tol <- sqrt(.Machine$double.eps)
step <- n/(svd(t(x)%*%x)$d[1])
```

```r
while(cvg[iter+1] > 1e-2 && iter < 100){
  iter <- iter + 1
  b <- b.old + (1/n)*step*t(x)%*%(y - x%*%b.old)
  b <- sapply(b, soft, lambda*step)
  diff <- sum(abs(b-b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```r
plot_data <- data.frame(
  x = seq_along(cvg), value = as.numeric(cvg))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "t", y = "Error") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

```r
zero_index <- which(b.old == 0)[1]
plot_data <- data.frame(
  x = seq_along(b.old), value = as.numeric(b.old))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  geom_vline(xintercept = zero_index,
             linetype = "dashed", color = "#B65C35")
             +
  scale_x_continuous(breaks = c(
    zero_index, 50, 100, 150, 200)) +
  labs(x = "Index", y = "Components") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```


Error
4
3
2
1
0
0
10
20
30
40
50
t



思考：参数λ是如何影响β估计量的稀疏性的?

1.4 坐标下降法

- 坐标下降法 *(Coordinate descent)*: 在每一轮中对于多元变量的单个变量逐个更新。
- 具体来说，对于优化问题

$$x^* = \arg\min_{x \in \mathbb{R}^p} f(x),$$

我们在第 t 次迭代中依次更新第 j 个变量，$j = 1, \dots, p$:

$$\boxed{x_{t+1,j} = \arg\min_{x \in \mathbb{R}} f(x_{t+1,1}, \cdots, x_{t+1,j-1}, x, x_{t,j+1}, \cdots, x_{t,p}).}$$

一轮下来，我们得到一组新的变量

$$x_{t+1} = (x_{t+1,1}, \cdots, x_{t+1,p}).$$

收敛性：考虑可分目标函数

$$f(x_1, \dots, x_p) = g(x_1, \dots, x_p) + \sum_{j=1}^{p} h_j(x_j),$$

其中 g 可微且凸，h_j 为凸函数。在适当正则条件下，坐标下降法收敛（Tseng, 1988, 2001）。

- Lasso 具有这种可分结构。

文献：

Tseng, P. (1988). *Coordinate ascent for maximizing nondifferentiable concave functions*. Technical Report LIDS-P-1840, MIT.

Tseng, P. (2001). *Convergence of a block coordinate descent method for nondifferentiable minimization*. *Journal of Optimization Theory and Applications*, 109(3), 475–494.

例 1.2（续）：Lasso problem

固定 β_j 外的其他变量, 则目标函数为一个一维的 *Lasso* 问题

$$f(\beta_j) = \frac{1}{2n}\|(y - X_{-j}\beta_{-j}) - X_j\beta_j\|_2^2 + \lambda|\beta_j| + C,$$

这里 X_{-j} 为从数据阵 X 除去第 j 列得到的矩阵, β_{-j} 同理, C 为与 β_j 无关的常数。

- $f(\beta_j)$ 的最小值为 (how?)

$$\hat{\beta}_j = \mathcal{S}_{\frac{n\lambda}{\|X_j\|_2^2}}\left(\frac{(y - X_{-j}\beta_{-j})^\top X_j}{\|X_j\|_2^2}\right)$$

```r
# Generate data
set.seed(1)
n <- 100
p <- 8
x <- matrix(rnorm(n*p), n, p)
beta <- c(3, 1.5, 0, 0, 2, 0, 0, 0)
y <- x%*%beta + rnorm(n)

# The norm square of each column of data matrix X
xn <- apply(x^2, 2, sum)

# The soft-thresholding operator
soft <- function(x, lam){
  ifelse(abs(x)>lam, sign(x)*(abs(x)-lam), 0)
}
# Initialization
b.old <- b <- rep(0, 8)
cvg <- c(1)
```

```r
iter <- 0
lambda <- 0.1
tol <- sqrt(.Machine$double.eps)

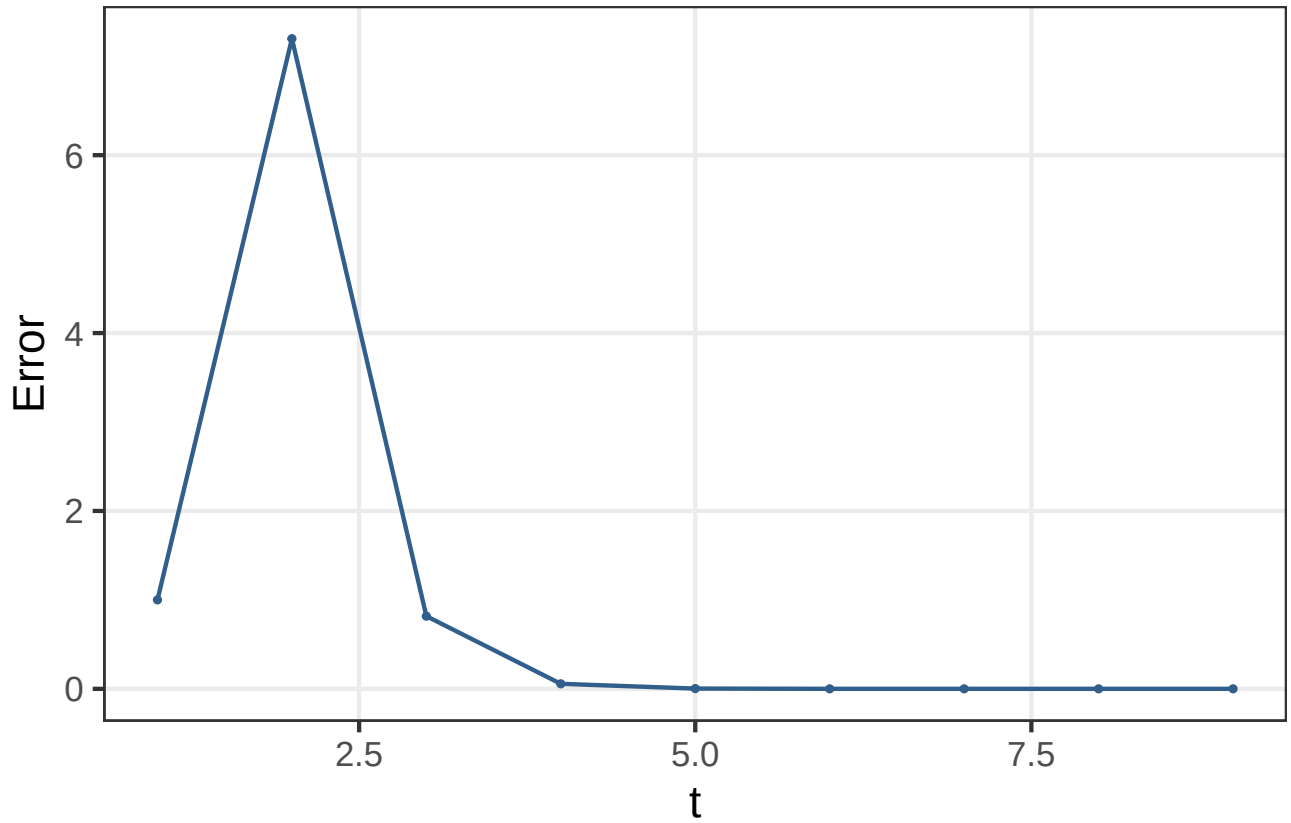
while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  for(j in 1:p){
    z <- y - x[,-j]%*%b[-j]
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])
  }
  diff <- sum(abs(b-b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```r
options(width = 50)
b.old # beta
```

```
## [1] 2.97028977 1.39565318 0.00000000 0.13616216
## [5] 2.01998563 0.07434669 0.00000000 0.00000000
```

```r
plot_data <- data.frame(
  x = seq_along(cvg), value = as.numeric(cvg))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  labs(x = "t", y = "Error") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

Note: 需要选择一个合适的λ

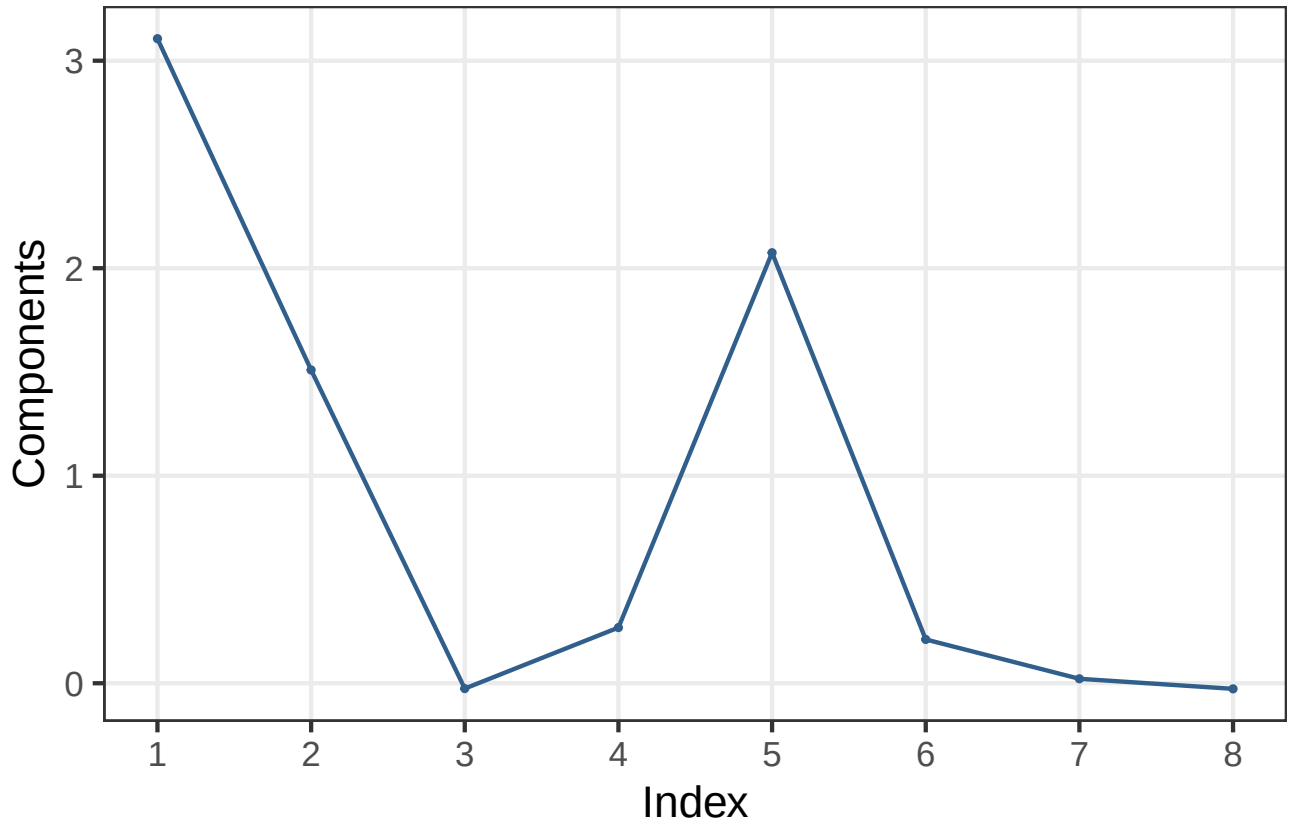

Error
6
4
2
0
2.5
5.0
7.5
t

- 取较小 λ

```
lambda <- 0.001
b.old <- b <- rep(0, 8)
cvg <- c(1)
iter <- 0
```

```
options(width = 50)
while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  for(j in 1:p){
    z <- y - x[,-j]%*%b[-j]
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])
  }
  diff <- sum(abs(b - b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```r
plot_data <- data.frame(
  x = seq_along(b.old), value = as.numeric(b.old))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  scale_x_continuous(breaks = 1:8) +
  labs(x = "Index", y = "Components") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

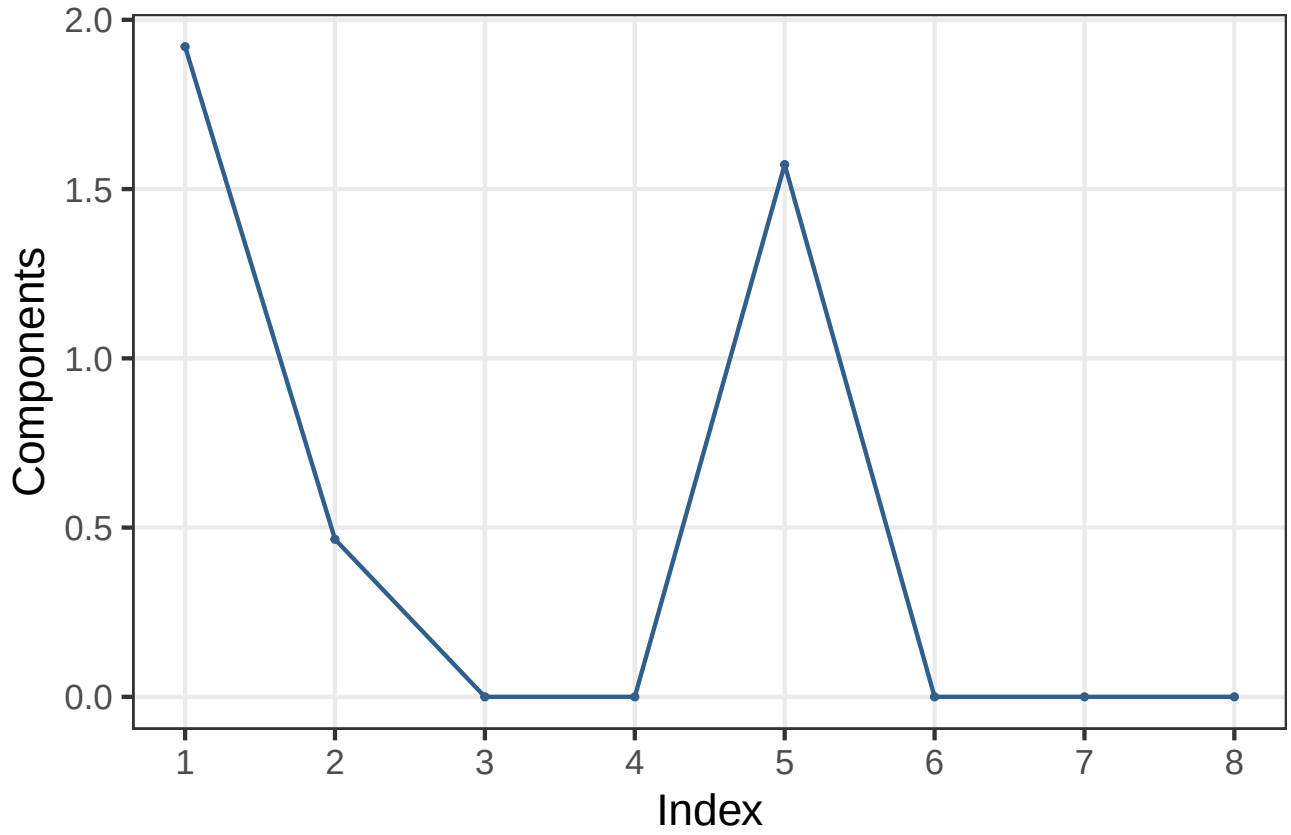

Components
3
2
1
0
1
2
3
4
5
6
7
8
Index

- 取较大 λ

```
lambda <- 1
b.old <- b <- rep(0, 8)
cvg <- c(1)
iter <- 0
```

```
while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  for(j in 1:p){
    z <- y - x[,-j]%*%b[-j]
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])
  }
  diff <- sum(abs(b - b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```r
plot_data <- data.frame(
  x = seq_along(b.old), value = as.numeric(b.old))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  scale_x_continuous(breaks = 1:8) +
  labs(x = "Index", y = "Components") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```

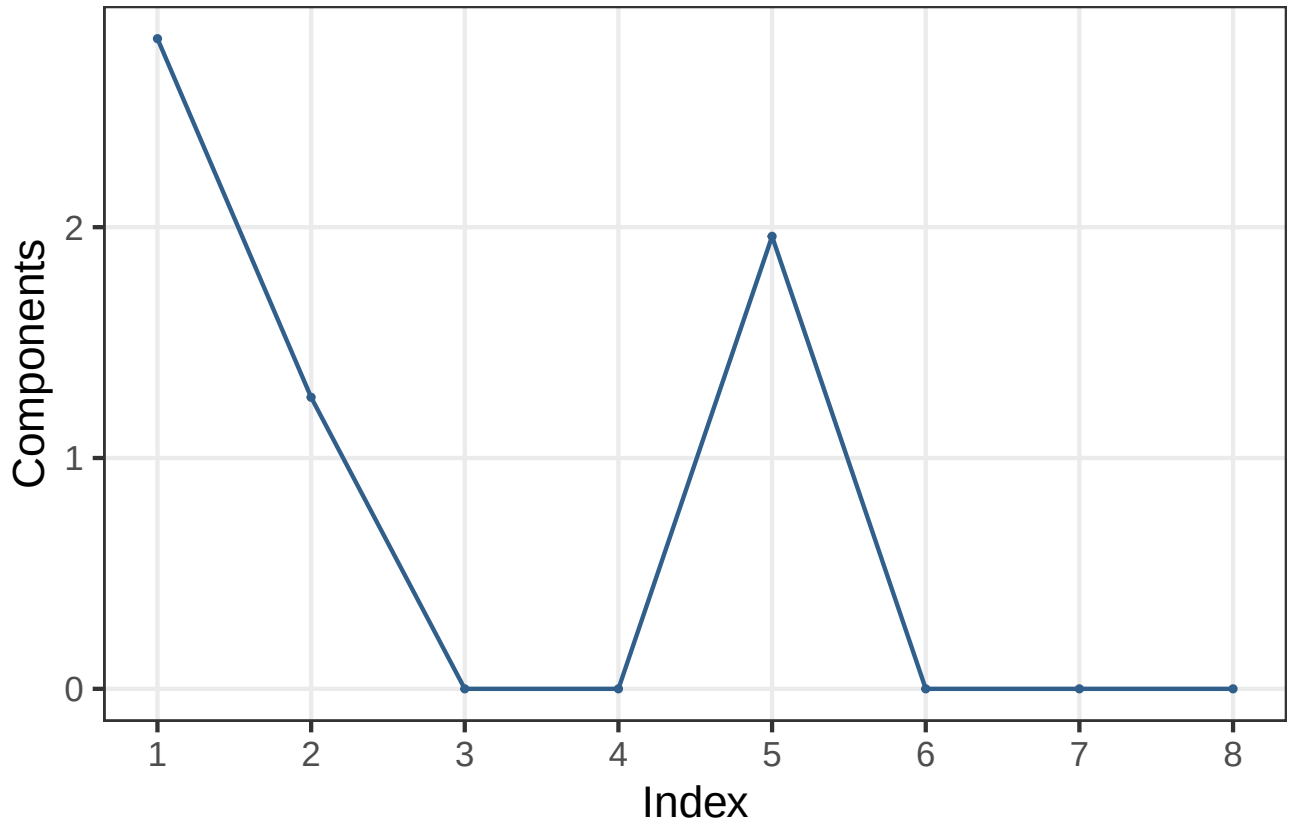

2.0
1.5
1.0
0.5
0.0
Components
1
2
3
4
5
6
7
8
Index

- 取合适 λ

```
lambda <- 0.22
b.old <- b <- rep(0, 8)
cvg <- c(1)
iter <- 0
```

```
while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  for(j in 1:p){
    z <- y - x[,-j]%*%b[-j]
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])
  }
  diff <- sum(abs(b - b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```r
plot_data <- data.frame(
  x = seq_along(b.old), value = as.numeric(b.old))
ggplot(plot_data, aes(x = x, y = value)) +
  geom_line(color = "#315F8C", linewidth = 0.65) +
  geom_point(color = "#315F8C", size = 0.7) +
  scale_x_continuous(breaks = 1:8) +
  labs(x = "Index", y = "Components") +
  theme_bw(base_size = 14) +
  theme(panel.grid.minor = element_blank())
```


Components
2
1
0
1
2
3
4
5
6
7
8
Index